Proposed System Scope & Boundaries

TIVIDY will be a generic workflow management system that stores reusable process descriptions and manages their execution. It will coordinate work, apply rules, track progress and maintain a history of actions. The selected scenario will use these capabilities without determining the entire architecture. The scope should be revised as we continue our design.

TIVIDY will:

1. Manage workflow definitions. Store descriptions of activities, their relationships, required information and process rules. Check that a definition is valid before starting an execution.
2. Manage workflow instances. Create an independent execution from a definition. Track its progress and determine when it has completed, been cancelled or failed.
3. Manage activities and dependencies. Represent individual activities and groups of activities. Support sequential and parallel work, and conditional routes. Rejected work may be reworked up to a configured retry limit. An activity becomes available only when its required conditions are satisfied.
4. Control valid actions. Track each work item's current state and allow only appropriate actions. Check that the participant requesting an action has permission to perform it.
5. Apply routing and assignment rules. Keep the decision about which activity happens next separate from the decision about who performs it. Allow these rules to change without rewriting the basic work-item classes.
6. Manage approvals and deadlines. Record approval outcomes, follow defined rejection or rework routes, and respond to missed deadlines. Apply escalation rules that transfer responsibility or request further intervention.
7. Handle events and external communication. Respond to relevant occurrences that affect progress. Communicate with external systems through defined interfaces that separate external-service details from the core engine.
8. Handle failures and maintain history. Distinguish expected business outcomes from technical failures. Apply configured retry or intervention rules. Record actions, outcomes and state changes without removing earlier history.
9. Provide monitoring and controlled recovery. Show current progress, waiting work, overdue activities and failures. Support reversal only where an action has a clearly defined recovery operation.

The engine is generic: it knows nothing about the industries with which it works. Each industry contributes a pack that supplies its workflow definition, its task implementations, its routing and assignment policies, its escalation chain, and its external adapters. No engine class contains an industry name or an industry-specific rule.

Information	Workflow Definition	Running Instance
Identity	Definition name, identifier and version	Instance identifier and the definition version being used

Information	Workflow Definition	Running Instance
Work structure	Activity descriptions, groups and connections	Activities created for this execution and their progress
Data	Required fields and validation rules	Actual values, results and associated records
Participants	Required roles and assignment rules	Eligible participants and actual assignments
Decisions	Conditions controlling routes and approvals	Decisions made and their outcomes
Timing and recovery	Deadline rules, retry limits and recovery options	Actual deadlines, attempts and failures
History	Descriptions of permitted activities and transitions	Recorded actions, actors, times and state changes

TIVIDY will coordinate activities and record their progress. Human participants and external systems will perform the work assigned to them. Process-specific calculations and rules will remain in activity implementations, definitions or policies.

The initial scope will include basic participant and role information, external-system interfaces and progress reporting. We propose deferring a graphical workflow editor, full BPMN compatibility, distributed execution, production identity services, analytics, and multi-version definition storage. The library holds one active definition per name; each instance records the version it was created from, preserving the definition-instance association the design requires.

Each instance must keep its own data, assignments, progress and history. Each work item must have a clear owner responsible for its creation and lifetime. Dependencies must be satisfied before work proceeds, and parallel activities must meet the agreed completion conditions before subsequent work begins.

Repeated or outdated events must not cause incorrect changes. Recovery actions must preserve the original history and create a record of the recovery performed. Retry logic may be maintained inline, rather than being a configurable strategy in order to maintain basicity of scope.

A work item may be created, made available, assigned, started, completed, rejected, cancelled or escalated. Operations which are valid in one state may be invalid in another, so TIVIDY must enforce valid transitions. Work items may acquire additional behaviour (such as validation, auditing and notifications (which we propose will both be handled at event level, one audit observer for the whole instance), etc.) without becoming a different kind of task; a direct approach can be taken to action validation by having it happen inside each state class.

Each workflow instance remains associated with the definition version it was created from Changes to a definition does not affect work already in progress. Running structures (instances, stages, work items, etc.) can be traversed for different purposes (such as monitoring, reporting, recovery) without exposing their internal storage to the rest of the system.